KG 구축 K2 엔티티와 관계 추출

추출 단위 하나와 그 문서 유형의 그래프 스키마를 LLM에 넣어, 스키마 모양대로 노드와 관계를 JSON으로 받음

LLM에 넣는 것

① 그래프 스키마 설계
문서유형별로 사전에 정의

② 메타데이터
예: record_id, 상담일, 채널, 주제, 가명 회원 키

③ 추출 단위 본문
예: 조항 1건, 상담 1건

④ 추출 지시
neo4j-graphrag라이브러리의 프롬프트 그대로 사용

LLM
gpt-oss-120b (Groq)
단위 하나에 1번
동시 4건

나온 그래프 (상담 C-20260501-003 예시)

Consultation C-20260501-003, 콜센터	HAS_PROPOSAL	**Proposal** 종이 영수증 대조 (거절)
Consultation C-20260501-003	HAS_PROPOSAL	**Proposal** 주문 내역 화면 대조 (수용)
Proposal 주문 내역 화면 대조	RECONSIDER_WHEN	**ReconsiderCondition** 다음 명세서 나올 때 다시 문의
Consultation C-20260501-003	FOLLOWS	**Consultation** C-20260302-002 (앞 상담)

1 같은 본문은 다시 묻지 않음
본문 해시와 스키마 지문이 같으면 앞 결과를 씀

2 형식 오류는 다시
같은 입력으로 2번까지 다시 물음

3 실패는 기록하고 계속
failed_units에 남기고 나머지 단위는 진행

4 스키마 밖은 K3(가지치기)단계에서 지움
스키마에 없는 노드, 관계, 속성은 바로 가지치기

무단전재 및 배포 금지

K2 추출 코드 (kg_build.py)

추출기는 한 번 만들고, 단위마다 머리줄을 붙여 부르며, 한 함수 안에서 K2 추출과 K3 가지치기를 함께 수행함

추출기 만들기와 _k2_k3_one (kg_build.py 발췌)

```
# prepare() 안: 추출기와 가지치기는 한 번만 만듦
llm = make_llm()                          # Groq openai/gpt-oss-120b
extractor = LLMEntityRelationExtractor(llm=llm,
    create_lexical_graph=False, on_error=OnError.RAISE)
pruner, sem = GraphPruning(), asyncio.Semaphore(config.LLM_CONCURRENCY)

async def _k2_k3_one(sem, extractor, pruner, unit, spec):   # K2와 K3를 함께
    async with sem:                        # 동시 4건
        text = _head_line(unit["metadata"], spec["header"]) + unit["text"]
        for attempt in range(config.LLM_RETRIES + 1):    # 2번까지 다시
            try:
                graph = await extractor.run(chunks=TextChunks(chunks=[
                    TextChunk(text=text, index=0, uid=unit["unit_id"])]),
                    schema=spec["schema"])
                break
            except Exception as e:
                if attempt == config.LLM_RETRIES:   # 실패는 기록용으로 돌려줌
                    return {"error": f"{type(e).__name__}: {str(e)[:200]}"}
        pruned = (await pruner.run(graph, spec["schema"])).graph   # K3: 추출 직후 바로
        return {"graph": pruned.model_dump(), ...}
```

LLM이 돌려준 그래프 (예시 상담, 일부)

```
{"nodes": [
  {"id": "c0", "label": "Consultation", ...},
  {"id": "p1", "label": "Proposal",
   "properties": {"content": "그러면 주문 내역 화면으로 ...",
                  "decision": "수용"}},
  {"id": "r0", "label": "ReconsiderCondition", ...}, ... ],
 "relationships": [
  {"type": "HAS_PROPOSAL",
   "start_node_id": "c0", "end_node_id": "p1"},
  {"type": "RECONSIDER_WHEN",
   "start_node_id": "p1", "end_node_id": "r0"}, ... ]}
```

앞 세대 결과 재사용 (prepare 발췌)

```
hit = cache.get((u["doc_type"], u["text_hash"], u["schema_signature"]))
if hit:
    u.update(graph=hit["graph"], reused=True)   # LLM을 부르지 않음
else:
    todo.append(u)
```

K2와 K3는 함께 수행 (_k2_k3_one 한 함수)
추출 직후 바로 가지치기한 뒤 kg_prepared.json에 보관

무단전재 및 배포 금지